

Introducción a la gestión y depuración de errores

Idea principal

Los errores son inevitables en la programación, incluso para expertos. Por eso, el **manejo de errores** y la **depuración** son habilidades esenciales para escribir programas en Python más robustos, confiables y fáciles de usar.

Manejo de errores (Excepciones)

- Funciona como una **red de seguridad** para el código.
- Permite anticipar problemas y evitar que el programa se bloquee de repente.
- Puede:
 - Detectar errores.
 - Mostrar mensajes informativos.
 - Tomar acciones correctivas.

Depuración

- Es el **trabajo detectivesco** de la programación.
- Consiste en investigar el código de forma sistemática para:
 - Rastrear los errores.
 - Encontrar su causa.
 - Corregirlos paso a paso.
- Usa herramientas que permiten:
 - Ejecutar el código línea por línea.
 - Examinar variables.

- Localizar exactamente dónde ocurre el fallo.

Beneficios

- Programas más **confiables** y **robustos**.
- Mejor experiencia para el usuario (el programa no se rompe sin explicación).
- Permite afrontar proyectos más complejos con **confianza**.
- Te convierte en un **programador más competente**.

Excepciones de Python parte 1: Entender las banderas rojas

Idea principal

Las **excepciones** en Python funcionan como **alarmas** que avisan cuando ocurre algo inesperado durante la ejecución, evitando bloqueos y permitiendo al programador manejar el problema de forma controlada.

Qué son las excepciones

- Interrumpen el flujo normal del programa.
- Actúan como la **luz de advertencia** de un coche: avisan antes de que ocurra un fallo grave.
- Se generan en situaciones como:
 - Dividir entre cero.
 - Acceder a un archivo inexistente.
 - Introducir un dato inválido.

Objeto de excepción

Cuando se produce un error, Python crea un objeto con:

- **Tipo de excepción:** naturaleza del error (ej. `ZeroDivisionError`, `FileNotFoundError`).
- **Rastreo (traceback):** historial de las llamadas que llevaron al fallo, muestra dónde ocurrió.
- **Mensaje de error:** texto explicativo con detalles para entender y resolver el problema.

Manejo de excepciones (try-except)

- Se utiliza para **detectar y gestionar errores** sin que el programa se bloquee.
- **Bloque `try`:** contiene el código que podría fallar.
- **Bloque `except`:** red de seguridad, define cómo actuar si ocurre la excepción.
- Se pueden tener **varios `except`** para tratar distintos errores con mensajes personalizados.

Ejemplo básico:

```
try:
    resultado = 10 / 0
except ZeroDivisionError:
    print("Error: No se puede dividir entre cero.")
```

Buenas prácticas

- Usar excepciones **solo para condiciones inesperadas**, no como método para controlar el flujo normal del programa.
- Anticipar posibles errores y prevenirlos siempre que sea posible.

En conclusión, las excepciones permiten escribir programas más **seguros, robustos y fáciles de usar**, al dar tiempo al programador para responder a los errores antes de que el código falle por completo.

Pregunta

¿Qué información contiene un objeto de excepción? Seleccione todo lo que corresponda.

☒ Un rastreo

☒ Correcto

Correcto Los objetos de excepción contienen el tipo de excepción, un rastreo y el mensaje de error para ayudarle a localizar y resolver el problema.

☐ Frecuencia del error

☒ Tipo de excepción

☒ Correcto

Correcto Los objetos de excepción contienen el tipo de excepción, un rastreo y el mensaje de error para ayudarle a localizar y resolver el problema.

☒ El mensaje de error

☒ Correcto

Correcto Los objetos de excepción contienen el tipo de excepción, un rastreo y el mensaje de error para ayudarle a localizar y resolver el problema.

Excepciones de Python parte 2: Anatomía de una excepción

Idea central

Las **excepciones** no son solo interrupciones molestas: ofrecen una forma estructurada de manejar errores, mejorar la experiencia del usuario, facilitar la depuración y aumentar la mantenibilidad del código.

Beneficios de las excepciones

1. Gestión de errores:

- Evitan que el programa se bloquee.
- Permiten dar mensajes claros e incluso sugerencias (ej. “verifica la ruta del archivo”).

2. Experiencia del usuario:

- Ofrecen mensajes informativos o amigables que ayudan a resolver el problema.
- Ejemplo: validar una dirección de correo electrónico en un formulario.

3. **Depuración:**

- Actúan como “migas de pan” que señalan dónde ocurrió el error.
- El rastreo y el mensaje de error ayudan a identificar y corregir fallos rápidamente.

4. **Mantenibilidad:**

- El manejo de excepciones hace que el código sea más robusto, predecible y fácil de mantener en proyectos grandes.

Excepciones comunes en Python

- **ZeroDivisionError**: dividir entre cero.
- **FileNotFoundError**: archivo inexistente o dañado.
- **TypeError**: operación con tipos incompatibles (ej. sumar cadena + número).
- **ValueError**: argumento de tipo correcto pero con valor inapropiado (ej. `int("hola")`).
- **IndexError**: índice fuera de rango en listas o secuencias.

Otras excepciones notables

- **ImportError**: módulo no encontrado.
- **KeyError**: clave inexistente en un diccionario.
- **AttributeError**: atributo no válido en un objeto.
- **SyntaxError**: error en la sintaxis del código.

Aplicaciones prácticas

- **Validación de datos:** detectar entradas inválidas como correos mal formados.
- **Operaciones de red:** manejar errores de conexión, tiempos de espera y reintentos.

En resumen, **comprender y manejar excepciones** permite crear aplicaciones Python más **confiables, fáciles de usar y resistentes a errores del mundo real**.

Pregunta

Estás desarrollando un script en Python que lee datos de archivos externos y realiza cálculos. Durante las pruebas, se encuentra con una situación en la que el script intenta acceder a un archivo que falta en el directorio especificado. ¿Cuál de los siguientes tipos de excepción incorporados es MÁS probable que se produzca en este caso? Seleccione la mejor respuesta.

- ☐ `TypeError`
- ☐ `ZeroDivisionError`
- ☒ `FileNotFoundError`
- ☐ `ValueError`

✔ **Correcto**

Correcto: `FileNotFoundError` se activa específicamente cuando no se puede encontrar un archivo en la ruta especificada, de acuerdo con el escenario de la pregunta.

Excepciones comunes de Python: Una guía de campo

Contexto

Las **excepciones** son inevitables en programación. No gestionarlas puede provocar caídas, pérdida de datos y mala experiencia de usuario. Comprenderlas y manejarlas correctamente es esencial para construir aplicaciones robustas.

1. NameError

Se produce cuando se intenta usar una variable, función o módulo **que no ha sido definido o importado**.

Causas principales

- **Errores tipográficos** (Python distingue mayúsculas/minúsculas).
- **Problemas de alcance (scope)**: acceder a una variable fuera de su función.
- **Falta de importación de módulos**.

Ejemplo de scope

```
def my_function():  
    local_var = 10  
  
print(local_var)  
  
# NameError: name 'local_var' is not defined
```

Ejemplo de importación faltante

```
print(sqrt(9))  
  
# NameError: name 'sqrt' is not defined
```

Soluciones

- Revisar nombres cuidadosamente.
- Entender las reglas de **variables globales y locales**.

Importar correctamente los módulos:

```
import math  
  
print(math.sqrt(9))
```

-

En producción, un **NameError no gestionado** puede cerrar la aplicación, frustrar al usuario y hacer perder datos.

2. TypeError

Aparece al intentar combinar o manipular datos de **tipos incompatibles**.

Causas principales

- **Operaciones inválidas** (ej. sumar cadena + entero).
- **Argumentos de función incorrectos**.
- **Problemas en herencia de clases** (métodos incompatibles con la clase padre).

Ejemplo de operación inválida

```
print("hello" + 5)  
  
# TypeError: can only concatenate str (not "int") to str
```

Ejemplo de argumentos incorrectos

```
print(len(123))  
  
# TypeError: object of type 'int' has no len()
```

Ejemplo de verificación en función

```
def calculate_area(length, width):  
    if not isinstance(length, (int, float)) or not isinstance(width,  
        (int, float)):  
        raise TypeError("Length and width must be numbers.")  
    return length * width  
  
print(calculate_area(5, 'three'))  
  
# TypeError: Length and width must be numbers.
```


Soluciones

Convertir datos al tipo correcto:

```
print(str(5) + " apples") # "5 apples"
```

-
- Usar `isinstance()` para validar tipos antes de operar.
- Diseñar bien jerarquías de clases, usando clases base abstractas si es necesario.

Un **TypeError no gestionado** puede generar comportamientos inesperados, corrupción de datos y bloqueos.

En resumen:

- **NameError** ocurre cuando Python no reconoce un nombre.
- **TypeError** aparece cuando se combinan tipos de datos incompatibles.
- Ambos errores son comunes y deben gestionarse con buenas prácticas: revisión cuidadosa, validación de tipos y uso correcto de `import`.

IndexError: La confusión con los índices

Ocurre cuando se intenta acceder a una **posición inválida** en una lista, tupla o cadena.

Causas principales

- Acceder a un índice **fuera de rango**.
- Intentar acceder a un índice en una **secuencia vacía**.

Ejemplo de acceso fuera de rango

```
my_list = [1, 2, 3]

print(my_list[3])

# IndexError: list index out of range
```

Ejemplo de acceso seguro con bucle

```
my_list = [1, 2, 3]

for i in range(len(my_list)):

    print(my_list[i])

# 1

# 2

# 3
```

Ejemplo con try-except para listas vacías

```
my_list = []

try:

    print(my_list[0])

except IndexError:

    print("The list is empty.")

# The list is empty.
```

En producción, un **IndexError no gestionado** puede cerrar el programa si los índices provienen de datos dinámicos (ej. entradas de usuario).

KeyError: La confusión con las claves

Aparece al intentar acceder a un **diccionario** usando una **clave inexistente**.

Causas principales

- Usar una clave que no existe.
- Creación dinámica de claves incorrectas.

Ejemplo de KeyError

```
my_dict = {"a": 1, "b": 2}

print(my_dict["c"])

# KeyError: 'c'
```

Ejemplo con try-except

```
my_dict = {"a": 1, "b": 2}

try:
    print(my_dict["c"])
except KeyError:
    print("Key not found in dictionary.")

# Key not found in dictionary.
```

Ejemplo con logging para producción

```
import logging

my_dict = {"a": 1, "b": 2}

try:
    print(my_dict["c"])
except KeyError as e:
    logging.error(f"KeyError encountered: {e}")

    # Handle the error or provide a fallback mechanism
```

En producción, un **KeyError no gestionado** puede causar bloqueos o resultados inesperados, por ejemplo al buscar claves inexistentes en bases de datos.

Buenas prácticas con excepciones

1. **Leer el mensaje de error con atención** → indica tipo, línea y descripción.
2. **Diagnosticar la causa raíz** → tipografía, scope, índice inválido o clave inexistente.
3. **Probar soluciones** → condicionales, comprobaciones (`in`, `len`, `.get()`), o bloques `try-except`.
4. **Registrar errores en producción** para análisis posterior.

Cada excepción es una **oportunidad de aprendizaje** que ayuda a escribir código más robusto, estable y mantenible.

Reto de código funcional: Manejar una `KeyError`

```
def get_city_population(populations, city):  
    try:  
        return populations[city]  
    except KeyError:  
        # Lanzamos un KeyError con un mensaje personalizado  
        raise KeyError(f'City "{city}" not found in population  
data.')
```

Ejemplo 1: Ciudad inexistente

```
city_populations = {"New York": 8336817, "Los Angeles": 3979576,  
"Chicago": 2679044}  
  
city_name = "Tampa"
```

```
try:

    print(get_city_population(city_populations, city_name))

except KeyError as e:

    print(e)

# Ciudad "Tampa" no encontrada en los datos de población.
```

Ejemplo 2: Ciudad existente

```
city_name = "New York"

print(f"The population of {city_name} is {city_populations["New
York"]} ") # accedemos por clave no por índice

print(f"The population of {city_name} is
{get_city_population(city_populations, city_name)}") #otra forma
```

'City "Tampa" not found in population data.'

The population of New York is 8336817

The population of New York is 8336817

Entender la función

```
def get_city_population(populations, city):

    return populations[city]
```

```
city_populations = {"New York": 8336817, "Los Angeles": 3979576,
"Chicago": 2679044}
```

```
city_name = "Chicago"
```

```
print(get_city_population(city_populations, city_name))
```

2679044

Cuestionario

1. Estás trabajando con un nuevo desarrollador en un proyecto y está confundido sobre el propósito del manejo de errores. Te preguntan: "¿Por qué debería molestarme con la gestión de errores? Parece mucho código extra para algo que puede que ni siquiera ocurra" ¿Cómo explicarías la importancia del tratamiento de errores? Selecciona la mejor respuesta.

- ☐ El tratamiento de errores encuentra y corrige automáticamente todos los fallos del código.
- ☒ El tratamiento de errores ayuda a evitar que el programa se bloquee inesperadamente y lo mantiene funcionando sin problemas incluso cuando algo va mal.
- ☐ El tratamiento de errores hace que su código se ejecute más rápido porque puede ignorar los errores.
- ☐ El tratamiento de errores permite probar el código sin tener que depurarlo manualmente.

✓ **Correcto**

Corregir errores El tratamiento de errores permite a su programa anticipar y manejar con elegancia situaciones inesperadas, asegurando la estabilidad.

2. ¿Cuáles de las siguientes son ventajas de la depuración en el proceso de desarrollo? Seleccione todas las que procedan.

☐ Elimina la necesidad de revisar el código.

☒ Ayuda a comprender el comportamiento y la lógica del código.

☒ **Correcto**

Correcto La depuración ayuda a comprender cómo se comporta el código y su lógica.

☒ Facilita la colaboración entre los miembros del equipo de desarrollo.

☒ **Correcto**

Correcto La depuración puede ayudar a los miembros del equipo a entender el código de los demás y a trabajar juntos para resolver los problemas.

☒ Mejora la calidad del código identificando y corrigiendo errores.

☒ **Correcto**

Correcto La depuración ayuda a mejorar la calidad del código identificando y resolviendo errores.

3. Estás revisando el código Python de un desarrollador junior y te das cuenta de que no ha incluido ningún tratamiento de errores. Cuando ejecutas el script, se bloquea con un mensaje que menciona una variable indefinida. ¿Qué excepción específica causaría este tipo de fallo? Selecciona la mejor respuesta.

☐ ValueError

☒ ErrorNombre

☐ KeyError

☐ TypeError

☒ **Correcto**

Correcto Se produce un NameError cuando se intenta utilizar una variable que aún no ha sido definida.

4. ¿Qué excepciones se pueden producir al manejar archivos en Python? Selecciona la mejor respuesta.

☐ Error de índice

☐ KeyError

☐ TypeError

☒ FileNotFoundError

☒ **Correcto**

Correcto FileNotFoundError se produce cuando se solicita un archivo o directorio que no existe.

5. Cuando se produce una excepción en Python, ¿qué sucede con el flujo normal de ejecución del código? Selecciona la mejor respuesta.
- ☐ El programa ignora la excepción y continúa ejecutando las siguientes líneas de código.
 - ☐ El programa deja de ejecutarse inmediatamente en la línea donde se produjo la excepción y salta al siguiente bloque de código.
 - ☐ El programa sigue ejecutando el código línea por línea hasta llegar al final del script.
 - ☒ El programa deja de ejecutarse inmediatamente en la línea donde se produjo la excepción y comienza a ejecutar el manejador de excepciones más cercano.

✓ Correcto

Correcto Cuando se produce una excepción, se ejecuta el manejador de excepciones más cercano.

Depuración de impresión: Tu fiel compañero

Depuración con `print` en Python

La **depuración de impresión** es una técnica sencilla que consiste en insertar sentencias `print()` en el código para mostrar valores y mensajes durante la ejecución. Esto permite rastrear el comportamiento del programa y detectar errores.

Usos principales

1. **Seguimiento de variables** → mostrar el valor actual de variables en distintas etapas.
2. **Entrada y salida de funciones** → marcar el inicio y fin de la ejecución de una función.
3. **Iteración en bucles** → mostrar el número de iteración y seguir el progreso del ciclo.
4. **Controles condicionales** → imprimir si se cumple o no una condición `if`.
5. **Aislamiento de errores** → colocar prints alrededor de secciones sospechosas.

Técnicas avanzadas

- **Salida formateada:** usar f-strings o `format()` para obtener mensajes claros y estructurados.
- **Impresión condicional:** mostrar mensajes solo cuando se cumplan ciertas condiciones.
- **Registro (logging):** en proyectos grandes, usar el módulo `logging` para generar registros con niveles (debug, info, warning, error) y marcas de tiempo.

Proceso paso a paso

1. Identificar secciones sospechosas del código.
2. Insertar `print()` en lugares estratégicos.
3. Ejecutar el código y observar la salida.
4. Analizar los resultados impresos.
5. Identificar la causa del error.
6. Corregir, probar de nuevo y refinar el uso de `print()` hasta resolver el problema.

En conclusión: la depuración con `print` es una forma simple y efectiva de encontrar errores, comprobar valores y entender el flujo del programa antes de que los problemas se vuelvan más graves.

Pregunta

Estás trabajando en un script de Python que automatiza el procesamiento de datos de varias fuentes. Has notado resultados inesperados en la salida final. ¿Cuál de las siguientes técnicas de depuración de impresión sería tu primer paso para identificar la posible fuente del error? Seleccione la mejor respuesta.

- ☐ Utilice el módulo `logging` para crear registros detallados de la ejecución de su programa, incluyendo marcas de tiempo y niveles de registro.
- ☐ Implemente la impresión condicional para mostrar mensajes sólo cuando se cumplan condiciones específicas, reduciendo potencialmente la sección de código problemática.
- ☒ Inserte estratégicamente sentencias `print` en puntos clave de su código para realizar un seguimiento de los valores de las variables y del flujo de ejecución.
- ☐ Utilice la salida formateada para presentar los resultados finales de una manera visualmente más atractiva y estructurada.

🟢 Correcto

Correcto Es una forma sencilla pero eficaz de conocer el comportamiento del programa e identificar el origen de los resultados inesperados.

Depuradores interactivos: recorre tu código

Uso de un depurador en lugar de `print`

- **Problema inicial:** el programador intentaba encontrar un error en el cálculo de un descuento usando `print()`, pero la salida era confusa y desordenada.
- **Comparación:**
 - Usar `print` → como buscar con una linterna en una casa.
 - Usar un **depurador** → como encender todas las luces: permite ver todo de forma clara.
- **Funcionamiento del depurador:**
 - **Establecer un punto de interrupción** en la línea sospechosa (aparece un punto rojo en el IDE).
 - Ejecutar el código en **modo depuración**: se detiene en el punto marcado.
 - Revisar valores de las variables y avanzar **línea por línea** con el botón de paso.
- **Ejemplo práctico:**
 - El error se debía a que se usaba `"0,1"` (cadena) en lugar de `0.1` (número).

Con el depurador se corrigió fácilmente y se comprobó que la línea de retorno debía ser:

```
return total - discount
```

-
- **Ventajas del depurador:**
 - Ahorra tiempo y frustración.
 - Permite comprender mejor cómo funciona el código.
 - Más ordenado y claro que imprimir todo manualmente.

Conclusión: aunque los `print()` son útiles, aprender a usar el depurador integrado del IDE facilita mucho la detección y corrección de errores.

Conjunto de herramientas de depuración en Python

La depuración es esencial para los desarrolladores porque permite detectar y corregir errores que podrían causar fallos, pérdidas de datos o incluso problemas de seguridad. Un ejemplo real fue el apagón de **Facebook en 2019**, causado por un error de configuración del servidor que generó pérdidas millonarias y disminuyó la confianza de los usuarios.

Dominar las técnicas de depuración no solo soluciona problemas inmediatos, sino que fortalece el software para hacerlo más **fiable, seguro y resistente**.

Herramientas y técnicas de depuración

1. Sentencias `print()`: El narrador del código

- Útiles para **ver valores de variables** y seguir el flujo de ejecución en **scripts pequeños o problemas rápidos**.
- Pueden volverse engorrosas en proyectos grandes.

Ejemplo inicial sin depuración:

```
def calculate_average(numbers):  
    total = sum(numbers)  
    count = len(numbers)  
    return total / count
```

Ejemplo con `print()` para depuración:

```
def calculate_average(numbers):  
    print("Input numbers:", numbers)  
    total = sum(numbers)  
    print("Total:", total)  
    count = len(numbers)  
    print("Count:", count)
```

```
average = total / count
print("Average:", average)
return average
```

Esto muestra los valores paso a paso y permite localizar errores fácilmente.

2. Registro (**logging**): Una crónica detallada

- Más potente que `print`, pensado para **programas grandes o de larga duración**.
- Permite guardar mensajes en **archivos** o **bases de datos**.
- Los mensajes se clasifican por niveles: `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`.
- Se pueden añadir marcas de tiempo y otros detalles.

Ejemplo con **logging**:

```
import logging

logging.basicConfig(filename='app.log', level=logging.DEBUG)
logging.debug('This is a debug message')
logging.info('This is an informational message')
logging.warning('This is a warning message')
logging.error('This is an error message')
```

Ventajas del registro sobre `print()`:

- Los mensajes se almacenan y se pueden consultar más tarde.
- La salida es más estructurada (con marcas de tiempo y metadatos).
- Se puede ajustar el nivel de detalle según las necesidades.

El **logging** es ideal en **sistemas complejos** y para seguir el historial de eventos de una aplicación.

Conclusión:

- Usa `print()` para problemas rápidos y sencillos.
- Usa `logging` para programas más grandes, largos o críticos, donde necesites **historial, estructura y flexibilidad** en la depuración.

Técnicas avanzadas de depuración en Python

La depuración es clave en el desarrollo de software: ayuda a **detectar errores**, refinar el código y mejorar la comprensión de Python. Varias herramientas y enfoques trabajan en conjunto para lograrlo.

1. Aserciones

Las **aserciones** son condiciones que se esperan cumplir en el código. Si fallan, lanzan una excepción inmediatamente, ayudando a detectar errores tempranos.

Ejemplo:

```
def calculate_area(length, width):  
    assert length > 0, "Length must be positive"  
    assert width > 0, "Width must be positive"  
    return length * width
```

2. Depuradores

Los **depuradores** permiten pausar la ejecución, revisar el estado de las variables y recorrer el código línea a línea.

Son ideales para:

- Localizar la línea exacta del error.
- Entender flujos de datos complejos.
- Probar soluciones en tiempo real.

3. Recursos en línea

La comunidad (GitHub, foros, documentación) es valiosa para:

- Encontrar soluciones ya probadas.

- Recibir enfoques alternativos.
- Aprender de discusiones técnicas.

Consejo: al pedir ayuda, proveer un fragmento claro de código, mensajes de error y pasos ya intentados.

4. Estudio de caso: Fuga de memoria en Flask

Una aplicación web en Flask sufría ralentización debido a una **fuga de memoria**.

Proceso de depuración:

- **Registro:** rastrear uso de memoria, detectando problemas tras cargas de imágenes.
- **Depurador:** reveló que objetos de imagen no se liberaban correctamente.
- **Sentencias `print()`:** mostraron direcciones de memoria repetidas, confirmando el fallo.
- **Recursos en línea:** la comunidad indicó que era necesario cerrar explícitamente los archivos de imagen.

Solución:

```
from PIL import Image

with Image.open(image_path) as img:
    # Process the image
    img.close() # Explicitly close the image file
```

Resultado: El rendimiento se estabilizó y desapareció la fuga de memoria.

Conclusión:

La depuración combina **aserciones, depuradores, logging, print y recursos en línea**. No es solo corregir errores, sino **refinar el código, comprender mejor Python y crecer como desarrollador**.

Importancia y beneficios de los depuradores en Python. El por qué y el cómo

Los **depuradores** son herramientas clave en la programación de Python porque permiten encontrar y corregir errores de manera rápida y precisa, evitando que se conviertan en problemas mayores.

Beneficios principales:

1. **Identificación rápida de errores:** en lugar de revisar el código manualmente o usar `print()`, los depuradores localizan la causa raíz en menos tiempo.
2. **Visión detallada del código:** muestran la ejecución paso a paso, permiten inspeccionar variables, probar valores temporales y establecer puntos de interrupción.
3. **Precisión:** ayudan a detectar errores sutiles difíciles de ver con técnicas manuales.
4. **Mejor comprensión del código:** al observar cómo fluye la lógica, se facilita predecir y prevenir errores futuros.
5. **Beneficio profesional y de proyecto:** mejoran la calidad del software, aumentan la satisfacción del usuario y convierten al programador en un desarrollador más eficiente.

Herramientas de depuración en Python:

- **PDB:** depurador integrado en línea de comandos.
- **Depuradores de IDEs:** como los de VS Code o PyCharm, con interfaces gráficas fáciles de usar.
- **Módulo logging:** registra eventos y errores para analizarlos después.
- **Depuración con print():** útil para comprobaciones rápidas.

Buenas prácticas al depurar:

- Revisar primero errores obvios (sintaxis, tipográficos).
- Aislar el problema para enfocarse en la sección crítica del código.

- Usar puntos de interrupción estratégicamente.
- Recorrer el código con funciones del depurador (**step over**, **step into**, **step out**).
- Examinar valores de variables en distintos puntos.
- Consultar la documentación del depurador.

Conclusión: La depuración no solo resuelve errores, también mejora la calidad del proyecto y el aprendizaje del programador. Practicar con depuradores es esencial para convertirse en un buen desarrollador de Python.

Pregunta

Un desarrollador que trabaja en un gran proyecto Python con múltiples módulos interconectados tiene problemas para entender cómo fluyen los datos entre estos módulos y cómo interactúan las diferentes partes entre sí. ¿Qué técnica de depuración le proporcionaría una mejor comprensión? Selecciona la mejor respuesta.

- ☐ Revisar manualmente el código e intentar rastrear el flujo de ejecución y el flujo de datos leyendo el código.
- ☒ Utilizar un depurador dentro de un IDE para recorrer el código línea por línea, inspeccionar variables y observar las interacciones entre los distintos módulos.
- ☐ Insertar sentencias `print()` en varios puntos del código para mostrar los valores de las variables y seguir el flujo del programa.
- ☐ Utilizar el módulo de registro para registrar eventos y mensajes durante la ejecución.

☒ **Correcto**

Correcto El guión destaca que los depuradores son especialmente útiles para comprender bases de código complejas y observar cómo interactúan las distintas partes del programa.

ACTIVIDAD: DEPURACIÓN DE CÓDIGO

Escenario

Acabas de ser contratado como Analista de datos en una conocida empresa de recolección de insectos. Tu primera tarea es limpiar su extensa base de datos de insectos, que lamentablemente ha estado plagada de algunos molestos errores de introducción de datos (¡y puede que algunos bichos de verdad!).

A través de una serie de ejercicios interactivos, aprenderás a identificar y corregir errores e incoherencias comunes en los datos utilizando Python. Analizará los mensajes de error, utilizará estratégicamente las sentencias de impresión y aplicará sus conocimientos de Python para garantizar la exactitud e integridad de los datos de recolección de insectos. Al final de esta actividad, serás un campeón de la caza de insectos, ¡listo para enfrentarte a cualquier conjunto de datos desordenado con confianza y precisión!

Objetivo

El objetivo de esta actividad es dotarte de las habilidades esenciales de depuración en Python. Aprenderás a identificar, analizar y resolver errores comunes que pueden producirse durante la programación, como `FileNotFoundError`, `TypeError`, `IndexError`, `KeyError`, `AttributeError` y `ZeroDivisionError`.

A través de ejercicios prácticos y explicaciones claras, adquirirá la confianza necesaria para enfrentarse a código con errores, analizar mensajes de error de forma efectiva y aplicar las soluciones adecuadas para garantizar que sus programas Python se ejecutan sin problemas.

Paso 1: Encontrar el error `FileNotFoundError`

Empezaremos enfrentándonos a un error común cuando trabajamos con ficheros: el `FileNotFoundError`. Esto ocurre cuando Python no puede localizar el fichero que queremos utilizar.

- El código para importar pandas e intentar leer un Archivo CSV (con un error deliberado en el nombre del archivo) ya está proporcionado.
- Primero, ejecute la celda para activar el `FileNotFoundError`.
- Intente entender el error: te dice que falta el archivo.
- Solucione el error consultando los consejos de depuración. Si necesita ayuda, intente descomentar la línea de código corregida que especifica el nombre de archivo correcto (`'insect_collection.csv'`).
- Vuelva a ejecutar la celda para cargar correctamente los datos de insectos.

Paso 2: Entendiendo el `TypeError` con Enteros

Ahora, intentará obtener detalles específicos de sus datos de insectos, como el primer dígito del número de patas que tiene un insecto.

- Se proporciona el código para acceder al recuento de patas del primer insecto e intentar obtener su primer dígito (con un error deliberado).
- En primer lugar, ejecute la celda para activar `TypeError`.
- Intente entender el error: indica que estás intentando tratar un número como una lista, lo cual no es posible.
- Soluciona el error consultando los consejos de depuración. Si necesitas ayuda, prueba a descomentar la línea que convierte el contador de tramos en una cadena antes de acceder a su primer carácter.
- Vuelva a ejecutar la celda para ver la salida correcta.

Paso 3: Manejo del `TypeError` con Strings

En este paso, se encontrará con otro tipo de `TypeError` al trabajar con cadenas.

- Se proporciona el código para crear una sentencia sobre un insecto (con un error deliberado al combinar una cadena y un número).
- Primero, ejecute la celda para activar el `TypeError`.
- Intente comprender el error: pone de relieve que no puedes combinar directamente cadenas y números.
- Solucione el error consultando los consejos de depuración. Si necesitas ayuda, intenta descomentar la línea que utiliza una cadena `f` para incrustar limpiamente el número dentro de la frase.
- Vuelva a ejecutar la celda para generar la sentencia corregida.

Paso 4: Lidiando con el `IndexError`

A continuación, practicará el acceso a la información sobre un insecto en una posición específica de su lista de datos de insectos.

- Se proporciona el código para intentar acceder a un insecto más allá de los límites de la lista (provocando un `IndexError`).
- Primero, ejecute la celda para provocar el `IndexError`.
- Intente comprender el error: significa que está intentando acceder a un elemento que no existe en la lista.
- Soluciona el error consultando los consejos de depuración. Si necesitas ayuda, prueba a descomentar las líneas que comprueban si el índice deseado es válido antes de acceder a él.
- Vuelva a ejecutar la celda para ver la salida o el mensaje correspondiente.

Paso 5: Solucionar el KeyError

En este paso, intentará acceder a información (color) que no está presente en los datos de su insecto.

- Se proporciona el código para intentar acceder al "color" de un insecto (que conduce a `KeyError`).
- En primer lugar, ejecute la celda para activar el `KeyError`.
- Intente comprender el error: significa que estás intentando utilizar una clave que no está en el diccionario.
- Soluciona el error consultando los consejos de depuración. Si necesitas ayuda, intenta descomentar las líneas que comprueban si existe la clave 'color' antes de intentar utilizarla.
- Vuelva a ejecutar la celda para obtener la salida o el mensaje correctos.

Paso 6: Superar el AttributeError

Ahora, intentará agregar la información de color a sus datos de insectos.

- Se proporciona el código para intentar añadir 'color' utilizando el método incorrecto (`.append()`, pensado para listas).
- Primero, ejecute la celda para activar el `AttributeError`.
- Intente entender el error: te dice que los diccionarios no tienen el método `.append()`.
- Soluciona el error consultando los consejos de depuración. Si necesitas ayuda, prueba a descomentar la línea que utiliza la forma correcta de añadir un par clave-valor a un diccionario.
- Vuelva a ejecutar la celda para ver los datos actualizados del insecto.

Paso 7: Manejo del ZeroDivisionError

Por último, calculará una relación entre las patas y las alas de los insectos, teniendo cuidado con los insectos con cero alas.

- Se proporciona el código para calcular la proporción entre patas y alas (con un potencial `ZeroDivisionError`).
- En primer lugar, ejecute la celda para activar el `ZeroDivisionError`.
- Intente comprender el error: se produce al intentar dividir por cero.
- Solucione el error consultando los consejos de depuración. Si necesita ayuda, intente descomentar las líneas que comprueban si hay alas cero antes de calcular el cociente.
- Vuelve a ejecutar la celda para ver las proporciones correctas o los mensajes para insectos sin alas.

1. ¿Cuál de los siguientes errores indica que se está intentando acceder a una clave que no existe en un diccionario? Seleccione la mejor respuesta.

- ☐ `TypeError`
- ☐ `AttributeError`
- ☒ `KeyError`
- ☐ `IndexError`

✓ **Correcto**

Correcto Un `KeyError` se plantea específicamente cuando se intenta acceder a una clave que no está presente en un diccionario.

2. ¿Cuál es el propósito principal de usar una sentencia `if` para comprobar si el divisor no es cero antes de realizar la división en Python? Seleccione la mejor respuesta.

- ☐ Para mejorar la legibilidad del código.
- ☐ Para optimizar el rendimiento del código.
- ☐ Para tratar los casos en los que el resultado de la división es cero.
- ☒ Para evitar que se produzca un `ZeroDivisionError`.

✓ **Correcto**

Correcto Dividir por cero no está definido en matemáticas, y Python lanza un `ZeroDivisionError` para evitarlo.

3. ¿Cuál de los siguientes métodos es la forma correcta de añadir un nuevo par clave-valor a un diccionario en Python? Selecciona la mejor respuesta.

- ☒ `dictionary[key] = value`
- ☐ `dictionary.append(key, value)`
- ☐ `dictionary.add(key, value)`
- ☐ `dictionary.insert(key, value)`

✓ **Correcto**

Correcto Esta es la forma estándar de añadir o actualizar pares clave-valor en un diccionario.

4. ¿Qué suele indicar la dirección `FileNotFoundError` en Python? Seleccione la mejor respuesta.

- ☒ El archivo al que intenta acceder no existe o no puede encontrarse en la ruta especificada.
- ☐ Está intentando acceder a un índice del archivo que está fuera de los límites.
- ☐ Está intentando realizar una operación en el archivo que no admite.
- ☐ Hay un desajuste entre el tipo de datos que está intentando leer del archivo y el tipo de datos esperado.

✓ **Correcto**

Correcto Esta es la causa más común de un `FileNotFoundError`.

Cuestionario

1. Estás depurando un script complejo de Python que se está comportando de forma inesperada. Has establecido un punto de interrupción en un punto crucial del código y ahora lo estás recorriendo usando el depurador de tu IDE. ¿Cuál de las siguientes acciones puede realizar típicamente mientras recorre su código con el depurador? Seleccione todas las que correspondan.

☒ Continúa la ejecución hasta el siguiente punto de interrupción.

☒ **Correcto**

Correcto Así es como te mueves eficientemente a través del código, deteniéndote sólo en los puntos de interés (breakpoints).

☒ Pasa por encima de las funciones para ejecutarlas sin entrar en ellas.

☒ **Correcto**

Correcto Pasar por encima de una llamada a una función le permite ejecutarla sin profundizar en sus detalles, lo que resulta útil cuando está seguro de que la función funciona correctamente.

☒ Inspecciona los valores de las variables en el punto actual de la ejecución.

☒ **Correcto**

Correcto Inspeccionar los valores de las variables es una parte fundamental de la depuración, ya que permite ver si son los esperados.

☐ Modifique directamente el código fuente mientras lo recorre.

2. Estás trabajando en un proyecto Python en tu IDE, y has encontrado un error inesperado durante la ejecución. Quieres examinar los valores de tus variables y el comportamiento del programa en un punto concreto del código. ¿Qué característica de tu IDE deberías utilizar para detener temporalmente la ejecución del programa en el punto deseado? Seleccione la mejor respuesta.

- ☐ Finalizar el programa
- ☐ Ejecutar un corrector sintáctico
- ☐ Comente la línea problemática
- ☒ Establecer un punto de interrupción

✓ **Correcto**

Correcto Establecer un punto de interrupción le permite detener el programa en la línea especificada e inspeccionar el estado del programa.

3. ¿Cuál de las siguientes es la finalidad principal del uso de aserciones en la programación en Python? Seleccione la mejor respuesta.

- ☐ Para mejorar el rendimiento del código.
- ☒ Validar la corrección del código durante el desarrollo.
- ☐ Proteger el código contra posibles amenazas a la seguridad.
- ☐ Para gestionar los errores en tiempo de ejecución con elegancia.

✓ **Correcto**

Correcto Las aserciones ayudan a validar la corrección del código y pueden detectar posibles errores en una fase temprana del proceso de desarrollo.

4. ¿Cuáles de las siguientes son estrategias efectivas para depurar en Python? Seleccione todas las que correspondan.

☒ Utilización de sentencias print para realizar un seguimiento de los valores de las variables.

☒ **Correcto**

Correcto Las sentencias Print pueden ayudarle a seguir el flujo del programa y el estado de las variables.

☒ Utilizar un depurador para recorrer el código.

☒ **Correcto**

Correcto Un depurador permite ejecutar código línea por línea e inspeccionar el estado en cada paso.

☒ Utilizar el registro A para registrar el flujo de ejecución del programa.

☒ **Correcto**

Correcto El registro A proporciona un registro de la ejecución del programa, que puede ser muy valioso para la depuración.

☐ Ignorar los errores menores para centrarse en los mayores.

5. Estás ayudando a un nuevo desarrollador de Python que está luchando por entender por qué su programa no está funcionando como se esperaba. Han oído que las sentencias print pueden ser útiles para depurar. ¿Cómo explicarías el principal beneficio de usar sentencias print en su código? Seleccione la mejor respuesta.

- ☐ Las sentencias Print encontrarán y corregirán automáticamente los errores de su código.
- ☒ Las sentencias de impresión pueden ayudarle a ver los valores de las variables y cómo se ejecuta el código paso a paso.
- ☐ Las sentencias de impresión eliminarán cualquier error de sintaxis de su código.
- ☐ Las sentencias Print harán que el programa se ejecute mucho más rápido.

☒ **Correcto**

Correcto Las sentencias Print permiten visualizar el estado del programa e identificar posibles problemas.

DetECCIÓN DE ERRORES CON try-except

Qué es

La estructura **TRY-EXCEPT-ELSE-FINALLY** permite **anticipar, detectar y manejar excepciones** para evitar que un programa se bloquee.

Bloques principales

- **TRY** → Contiene el código “arriesgado” que podría lanzar errores (operaciones de archivos, red, validación de entradas).
- **EXCEPT** → Captura y gestiona los errores; puede ser específico para distintos tipos de excepciones.
- **ELSE** → Opcional. Se ejecuta solo si no ocurre ninguna excepción, útil para indicar que el código se ejecutó con éxito.
- **FINALLY** → Siempre se ejecuta, ocurra o no error. Ideal para limpieza: cerrar archivos, liberar recursos, etc.

Usos comunes

- **Archivos** → Manejo de errores como archivo no encontrado o permisos restringidos.
- **Operaciones de red** → Manejar conexiones fallidas o tiempos de espera con reintentos.
- **Entrada de usuario** → Validar datos no válidos y mostrar mensajes claros en lugar de errores crípticos.

Beneficios

- Evita bloqueos inesperados.
- Mejora la experiencia del usuario con mensajes claros.
- Separa la lógica principal de la gestión de errores → código más organizado y legible.
- Facilita la depuración con rastreo y mensajes descriptivos.

- Hace el software más **robusto y fiable**, especialmente en proyectos grandes.

En resumen: **TRY-EXCEPT-ELSE-FINALLY** es una estrategia clave en Python para manejar errores de manera organizada, mejorar la usabilidad y crear programas más estables.

Pregunta

Estás escribiendo un script en Python para procesar datos de un Archivo CSV cargado por un usuario. Desea que el script gestione posibles errores, por ejemplo, si falta el archivo cargado o contiene datos no válidos. ¿Qué parte de la estructura `try-except-else-finally` sería la más adecuada para alertar al usuario si se produce una excepción durante el procesamiento del archivo?

- ☒ El bloque `except`
- ☐ El bloque `try`
- ☐ El bloque `else`
- ☐ El bloque `finally`

✔ Correcto

Correcto El bloque `except` está diseñado específicamente para capturar y gestionar excepciones, por lo que es el lugar ideal para mostrar un mensaje de error al usuario.

Demostración: Ejemplos de uso de excepciones en el mundo real (contiene código)

Importancia del manejo de excepciones

- El manejo de excepciones es como una **red de seguridad** que permite a las aplicaciones seguir funcionando aunque aparezcan errores.
- Garantiza que el programa no se bloquee bruscamente y que el usuario tenga una experiencia fluida.
- Python ofrece muchas excepciones incorporadas, y es recomendable atraparlas de forma **específica** para no ocultar otros problemas.

Estructura principal

- **TRY** → Contiene el código “arriesgado” que puede lanzar errores (ej. división, archivos, entradas).
- **EXCEPT** → Captura errores concretos y permite dar respuestas claras o alternativas.
- **ELSE** → Opcional. Se ejecuta solo si no ocurre ninguna excepción.
- **FINALLY** → Siempre se ejecuta, ocurra o no un error. Usado para limpieza (cerrar archivos, liberar recursos).
- **RAISE** → Permite **lanzar excepciones explícitas**, incluso personalizadas, útiles en dominios específicos de la aplicación.

Escenarios reales donde es crucial

1. **Archivos** → Manejo de archivos no encontrados, permisos o errores de lectura/escritura.
2. **Redes** → Control de tiempos de espera, errores de conexión, transmisión fallida.
3. **Entradas de usuario** → Validar datos inválidos o formatos inesperados y guiar al usuario.
4. **APIs externas** → Manejo de límites de velocidad, errores de autenticación, respuestas inesperadas.
5. **Bases de datos** → Resolver errores de conexión, conflictos de integridad o fallos de transacción.

Beneficios

- Evita bloqueos inesperados.

- Ofrece mensajes claros y amigables al usuario.
- Mejora la legibilidad y organización del código.
- Da resiliencia y fiabilidad a las aplicaciones.

En conclusión: **El manejo de excepciones en Python no solo soluciona errores, sino que construye aplicaciones robustas, adaptables y centradas en el usuario.**

Manejo de excepciones: Buenas prácticas

Importancia del manejo de excepciones

El manejo de excepciones es fundamental para crear **aplicaciones robustas y confiables**. Actúa como una red de seguridad que evita bloqueos inesperados y mejora la experiencia del usuario.

Razones clave para usarlo

1. Prevención de caídas abruptas

- Las excepciones no manejadas pueden cerrar un programa de golpe.
- Ejemplo: en un sitio de e-commerce, un fallo en el pago podría dejar al usuario sin pedido y sin confianza en la plataforma.

2. Recuperación gradual y mejor experiencia de usuario

- Permite mostrar mensajes claros, sugerir acciones alternativas o manejar el problema internamente.
- Ejemplo: si una API externa falla, se informa con un mensaje amigable en lugar de un error genérico.

3. Depuración optimizada

- Las excepciones ofrecen **rastreo detallado** que indica la línea y el origen del error.
- Ejemplo: un script de análisis de datos puede fallar por acceder a un índice fuera de rango; el rastreo muestra dónde ocurrió.

4. Profesionalidad y fiabilidad

- Refleja calidad y compromiso al prever y manejar errores.
- Ejemplo: una app financiera que gestiona entradas no válidas inspira más confianza que una que se bloquea con ellas.

En conclusión: **El manejo adecuado de excepciones previene errores críticos, mejora la depuración, protege la confianza de los usuarios y eleva la calidad del software.**

Manejo de excepciones en Python: try, except, finally

- **try:** Contiene el código que puede producir un error.
- **except:** Es el plan de contingencia; maneja el tipo de error específico.
- **finally:** Siempre se ejecuta; se usa para limpiar recursos (ej. cerrar archivos, conexiones).

Ejemplo: lectura de archivo con manejo de error

```
file_name = "sample.txt"
try:
    with open(file_name, 'r') as file:
        contents = file.read()
        print(contents)
except FileNotFoundError:
    print("Error: File not found -", file_name)
```

Excepciones comunes en Python

- **ZeroDivisionError** → división entre cero.

- **TypeError** → operación con tipos incompatibles.
- **FileNotFoundError** → archivo inexistente.
- **IndexError** → índice fuera de rango en listas o cadenas.
- **KeyError** → clave inexistente en un diccionario.

Ejemplo: manejo de múltiples excepciones

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Error: Division by zero")
except TypeError:
    print("Error: Invalid data types")
# Output: Error: Division by zero
```

Registro (logging) de errores

Permite guardar información sobre excepciones para depuración y análisis.

Ejemplo con logging

```
import logging

try:
    # Your potentially error-prone code here
    pass
except Exception as e:
    logging.error(f"An error occurred: {e}")
```

Excepciones personalizadas

Se pueden crear clases de error específicas para la aplicación.

Ejemplo

```
class InvalidCredentialsError(Exception):  
    pass  
  
if not valid_credentials(username, password):  
    raise InvalidCredentialsError("Incorrect username or password")
```

Evitar except demasiado genéricos

Capturar todas las excepciones oculta la causa real del error.

Ejemplo malo

```
try:  
    result = some_function()  
except:  
    print("An error occurred") # Oculta la causa real
```

Tipado de pato (duck typing) y manejo de errores

Python confía en el comportamiento de los objetos más que en su tipo estricto. Se deben manejar errores como `AttributeError`.

Ejemplo

```
def calculate_area(shape):  
    try:  
        return shape.calculate_area()  
    except AttributeError:  
        raise TypeError("Object does not have a calculate_area  
method")
```

Filosofías de manejo de errores

- **LBYL (Look Before You Leap)**: Comprobar condiciones antes de ejecutar.
- **EAFP (Easier to Ask Forgiveness than Permission)**: Ejecutar y atrapar errores (más Pythonico).

En resumen:

El manejo de excepciones en Python (con `try`, `except`, `finally`) permite crear programas más robustos, fáciles de depurar y confiables. Usar excepciones específicas, registro, excepciones personalizadas y la filosofía EAFP mejora la claridad y resiliencia del código.

Reto de código funcional: Lectura segura de archivos

```
def read_file_contents(file_path):
    try:
        with open(file_path) as file: # Intentamos abrir y leer el
archivo
            contents = file.read()
            print(contents)
    except FileNotFoundError:
        print("Error: File not found -", file_path)
```

Ejemplo de ruta

```
read_file_contents("/Users/Example/Documents/my_file.txt")
```

SALIDA:

```
Error: File not found - /Users/Example/Documents/my_file.txt
```

Cuestionario

1. ¿Cuál de las siguientes es una práctica importante a la hora de escribir manejadores de excepciones efectivos en Python? Seleccione la mejor respuesta.

- ☐ Plantear excepciones en bloques finally.
- ☒ Especifique claramente el tipo de excepción que está gestionando.
- ☐ Utilice una única cláusula except para gestionar todas las excepciones.
- ☐ Evite registrar excepciones para mantener el código limpio.

✓ **Correcto**

Correcto Especificar el tipo de excepción ayuda a capturar sólo las excepciones que desea manejar.

2. Cuáles son las prácticas que contribuyen a escribir manejadores de excepciones eficaces en Python. Selecciona todas las que correspondan.

☒ Atrapar sólo excepciones específicas.

✓ **Correcto**

Correcto La captura de excepciones específicas evita la gestión de excepciones no deseadas.

☐ Utilización de cláusulas de excepción desnudas.

☒ Generación de excepciones personalizadas.

✓ **Correcto**

Correcto Las excepciones personalizadas pueden proporcionar más contexto sobre el error.

☒ Registro de los detalles de la excepción.

✓ **Correcto**

Correcto El registro ayuda a diagnosticar y depurar problemas.

3. Eres mentor de un desarrollador junior de Python. Están trabajando en un script que necesita procesar datos de varios archivos. Algunos de estos archivos pueden faltar o estar corruptos. ¿Qué consejo le darías para asegurar que su script se ejecute sin problemas incluso si se encuentra con estos problemas? Selecciona la mejor respuesta.

- ☐ Comprueba dos veces todos tus cálculos matemáticos. Los errores aritméticos pueden dar lugar a resultados inesperados.
- ☐ Asegúrate de incluir muchas sentencias print a lo largo del código. Esto te ayudará a seguir el progreso de tu script y a detectar cualquier error.
- ☒ Envuelva siempre las operaciones con archivos, como la apertura y la lectura, dentro de un bloque try-except. De esta forma, si falta un archivo o no se puede leer, el script no se bloqueará.
- ☐ Organice su código en funciones bien definidas. Así será más fácil de leer y mantener.

✓ **Correcto**

¡Correcto! Manejo de excepciones es esencial cuando se trata de operaciones con archivos, ya que pueden surgir problemas inesperados.

4. ¿Cuál de los siguientes bloques de una estructura try-except-else-finally se ejecuta independientemente de si se ha producido una excepción o no? Seleccione la mejor respuesta.

- ☒ Finalmente
- ☐ Si no
- ☐ Pruebe
- ☐ Excepto

✓ **Correcto**

Correcto El bloque finally siempre se ejecuta, independientemente de si se ha lanzado o manejado una excepción.

5. ¿Cuál de los siguientes ejemplos requiere manejo de excepciones? Seleccione todo lo que corresponda.

☒ Conexión a una base de datos que puede estar desconectada.

☒ **Correcto**

Correcto Manejo de excepciones es crucial aquí para gestionar los posibles fallos de conexión.

☒ Acceso a un elemento de una lista por índice.

☒ **Correcto**

Correcto Manejo de excepciones ayuda a gestionar posibles errores de índice.

☒ Acceder a un diccionario con una clave que puede no existir.

☒ **Correcto**

Correcto Manejar las excepciones en este caso evita que los errores clave rompan el código.

☐ Concatenar dos cadenas.

Guía del detective para la depuración

La **depuración** es el proceso sistemático de identificar y corregir errores (*bugs*) en el código. No es un fracaso, sino un paso esencial para garantizar **funcionalidad, mejor experiencia de usuario y ahorro de tiempo/recursos**.

Propósitos principales:

1. **Funcionalidad:** asegura que el programa haga lo esperado sin fallar.
2. **Experiencia de usuario:** elimina frustración y confusión.
3. **Eficiencia:** resolver errores temprano es más barato y rápido.

Proceso estructurado de depuración (6 pasos):

1. **Reproducir el error** de forma consistente.
2. **Reunir pruebas:** mensajes de error, *stack trace*, comportamientos inusuales.
3. **Aislar la causa:** revisión de código, impresión de variables, uso de depurador.
4. **Hipotetizar y probar** posibles causas.

5. **Implementar la solución** adecuada.
6. **Probar la corrección** para confirmar que funciona y no introduce nuevos fallos.

Técnicas comunes:

- **Rubber Duck Debugging**: explicar el código línea por línea a un objeto.
- **Comentar código**: deshabilitar secciones para localizar problemas.
- **Pruebas unitarias**: detectar errores tempranos y evitar regresiones.

En definitiva, la depuración es una **habilidad esencial** que no solo corrige errores, sino que fortalece las capacidades del programador y mejora la calidad del software

Pregunta

Has encontrado un error en tu programa Python que hace que se bloquee inesperadamente. ¿Cuál de los siguientes pasos deberías seguir primero para depurar el problema? Selecciona la mejor respuesta.

- ☐ Modifique inmediatamente el código en la zona donde cree que puede estar el error.
- ☐ Consulte foros y comunidades en línea para ver si otros desarrolladores se han encontrado con un error similar.
- ☐ Revise el mensaje de error y la información de rastreo proporcionada por Python para conocer la naturaleza y la ubicación del error.
- ☒ Reproduzca el error, anotando los pasos o entradas específicos que lo desencadenan.

👉 **Correcto**

Correcto Reproducir el error crea un entorno controlado para la investigación, lo que le permite observar su comportamiento y reunir pistas esenciales para su posterior depuración.

Depuración en acción (**contiene código**)

La **depuración** es una habilidad esencial para todo programador, tanto principiante como experimentado. No se trata solo de corregir errores, sino de **entender mejor el código**, prevenir futuros problemas y fortalecer las habilidades de resolución de problemas.

Principios clave:

- La depuración es como resolver un **rompecabezas**: identificar, analizar y corregir errores sistemáticamente.
- Incluso errores menores (ej. una coma mal puesta o un tipo de dato incorrecto) pueden afectar a todo el programa.

- El primer paso es **reproducir el error** consistentemente para observarlo y recopilar pistas.
- Los **mensajes de error de Python** son una guía valiosa, ya que suelen señalar la causa.
- Un **depurador** permite pausar, recorrer el código línea por línea e inspeccionar variables con precisión.
- Tras corregir un error, es vital **probar el código con diferentes entradas y casos extremos**.
- Llevar un **diario de depuración** ayuda a documentar errores y soluciones para evitar repetirlos.

Escenario práctico:

Un programa para calcular promedios falla porque uno de los elementos de la lista es una cadena en lugar de un número.

1. El error se reproduce consistentemente.
2. Se insertan impresiones para rastrear variables.
3. El mensaje de error indica un problema de tipos.
4. Se filtran los valores no numéricos antes de calcular el promedio.
5. El código se corrige y se añaden pruebas unitarias.
6. Se documenta la experiencia en el diario de depuración.

Conclusión:

La depuración no es solo una técnica, es una **mentalidad**. Requiere paciencia, método y creatividad. Cada error es una **oportunidad de aprendizaje** que ayuda a crecer como programador y dominar el oficio.

Estrategias habituales de depuración en Python

La **depuración** es el proceso de identificar y resolver errores en el código. Es fundamental tanto para principiantes como para desarrolladores experimentados, ya que permite ahorrar tiempo, frustración y mejorar la eficiencia.

1. Descifrar mensajes de error

Los mensajes de error de Python ofrecen pistas clave:

- **Tipo de error:**
 - *SyntaxError*: errores de sintaxis (ej. `invalid syntax`).
 - *RuntimeError*: errores en ejecución, como `ZeroDivisionError` o `IndexError`.
 - *Errores lógicos*: más difíciles, producen resultados incorrectos aunque el código se ejecute.
- **Mensaje de error**: describe el problema concreto (ej. `NameError: name 'x' is not defined`).
- **Número de línea**: indica dónde se detectó el problema.

Ejemplo:

```
TypeError: 'str' object cannot be interpreted as an integer
```

→ Indica un uso incorrecto de tipos: una cadena donde se esperaba un entero.

2. Herramientas de depuración

- **Sentencias `print()`**: útiles para seguir el flujo y valores de las variables.
- **Depuradores (`pdb`, `ipdb`)**: permiten establecer *breakpoints*, inspeccionar variables en tiempo real y ejecutar el código paso a paso.
- **Aislamiento**: comentar o desactivar secciones del código para identificar la parte conflictiva.
- **Cuestionar suposiciones**: revisar la lógica y documentación de librerías externas.

- **Documentación y comunidades:** usar la documentación oficial de Python y foros como GitHub.

3. Ejemplo práctico: función de descuento

Versión inicial (incorrecta):

```
def calculatediscount(price, percentage):  
    if percentage < 0 or percentage > 100:  
        return "Invalid discount percentage" # Incorrect behavior  
    discountamount = price * (percentage / 100)  
    return price - discountamount  
  
print(calculatediscount(100, 150))  
  
# Devuelve "Invalid discount percentage" como cadena en lugar de  
# manejarlo como error.
```

Problema: la función devuelve un string en caso de error, lo cual no es la forma más robusta de señalar una condición inválida.

4. Solución mejorada: usar excepciones

```
def calculate_discount(price, percentage):  
    if percentage < 0 or percentage > 100:  
        raise ValueError("Discount percentage must be between 0 and  
100")  
    discount_amount = price * (percentage / 100)  
    return price - discount_amount
```

```
try:

    print(calculate_discount(100, 150))

except ValueError as e:

    print(f"Error: {e}")
```

Salida:

```
Error: Discount percentage must be between 0 and 100
```

Conclusión

El ejemplo demuestra cómo:

- Usar `print()` ayudó a detectar la entrada inválida.
- Reemplazar un retorno incorrecto por una excepción (`ValueError`) hizo el código más **robusto y claro**.

En definitiva, la depuración efectiva combina **lectura de errores, uso de herramientas, aislamiento de problemas, verificación de suposiciones y apoyo en la documentación/comunidades**.

Arsenal de depuración avanzado en Python

A medida que un programador gana experiencia, su arsenal de herramientas de depuración se amplía con técnicas más avanzadas que permiten resolver errores complejos, mejorar la calidad del código y optimizar el flujo de trabajo.

1. Control de versiones (Git)

- Actúa como una **máquina del tiempo del código**, registrando cada cambio.
- Permite **volver a versiones estables** si se introduce un error.
- Herramientas clave:

- `git log` → ver historial de confirmaciones.
- `git diff` → comparar versiones.
- `git checkout` → volver a un estado anterior.

Ejemplo: detectar una regresión en una función que antes funcionaba correctamente.

2. Desarrollo guiado por pruebas (TDD)

- Se escriben pruebas antes del código real.
- Garantiza que el código cumple los requisitos esperados y facilita la depuración.
- Ejemplo: pruebas para una función de ordenación deben verificar que:
 - Ordena enteros en orden ascendente.
 - Maneja listas vacías.
 - Ordena listas con duplicados.

3. Estructuras de registro (logging)

- Alternativa más avanzada a `print()`.
- Permite clasificar mensajes según **niveles de gravedad**: DEBUG, INFO, WARNING, ERROR, CRITICAL.
- Facilita priorizar y centralizar información en archivos o bases de datos.

Ejemplo de código:

```
import logging
```

```
logging.basicConfig(filename='myapp.log', level=logging.DEBUG)
```



```
def divide(x, y):  
    try:  
        result = x / y  
  
        logging.info(f"Successfully divided {x} by {y} to get  
{result}")  
  
        return result  
  
    except ZeroDivisionError:  
        logging.error("Division by zero attempted!")  
  
        return None  
  
divide(10, 2)  
  
divide(5, 0)
```

Salida en el archivo `myapp.log`:

INFO: Successfully divided 10 by 2 to get 5.0

ERROR: Division by zero attempted!

4. El Zen de la depuración

Más allá de herramientas, la depuración es una **mentalidad**:

- **Paciencia:** investigar sin prisas.
- **Método:** cambios planificados, no aleatorios.
- **Curiosidad:** entender el “por qué” del error, no solo el “cómo” corregirlo.
- **Aprendizaje:** documentar errores y soluciones para crecer como programador.

Conclusión:

Adoptar control de versiones, TDD, logging estructurado y una mentalidad positiva convierte la depuración en un **reto gratificante** y no en una tarea temida. Te hace un desarrollador más hábil, seguro y resiliente.

Bloque de código funcional: Depuración de un cálculo defectuoso

```
def calculate_discount(price, discount_percentage):  
    discount_amount = price * (discount_percentage / 100)  
    discounted_price = price - discount_amount  
    return discounted_price  
  
# Code to test your output  
  
price = 50  
discount_percentage = 20  
discounted_price = calculate_discount(price, discount_percentage)  
print(discounted_price)
```

salida: 40.0 (simplemente era cambiar de un + a un - en disconuted price)

Cuestionario

1. Al depurar un programa, ¿cuál de los siguientes pasos puede ayudarle a identificar la causa de un error? Seleccione todos los que correspondan.

☒ Utilizar un depurador para recorrer el código línea por línea.

☒ **Correcto**

Correcto Un depurador le permite ejecutar su código paso a paso, lo que puede ayudarle a comprender mejor el flujo e identificar dónde se producen los errores.

☐ Ignorar los mensajes de error y volver a ejecutar el programa.

☒ Añadir sentencias de impresión para comprobar los valores de las variables en diferentes puntos del programa.

☒ **Correcto**

¡Correcto! Las sentencias Print son una forma simple pero efectiva de seguir el estado de las variables y entender dónde pueden estar fallando las cosas.

☒ Comentar secciones del código y volver a ejecutar el programa.

☒ **Correcto**

Correcto Esta técnica puede ayudar a aislar la parte del código que causa el problema.

2. Mientras utiliza un depurador, observa que una variable x no mantiene el valor esperado. ¿Cuál de las siguientes podría ser la razón de este problema? Seleccione la mejor respuesta.

☐ La variable x es una variable global y se accedió a ella dentro de una función.

☒ La variable x no se ha inicializado correctamente antes de ser utilizada.

☐ La variable x es una cadena y se ha concatenado con un entero.

☐ La variable x se ha utilizado en un bucle.

☒ **Correcto**

Correcto Una variable mal inicializada puede provocar un comportamiento inesperado. Asegúrese de inicializar correctamente sus variables antes de utilizarlas.

3. ¿Cuáles de los siguientes son métodos eficaces para depurar un programa Python? Seleccione todos los que correspondan.

☐ Ignorar los mensajes de advertencia mientras se ejecuta el programa.

☒ Uso de sentencias print para mostrar valores de variables en diferentes puntos del programa.

☒ **Correcto**

Corregir Las sentencias de impresión pueden ayudarle a comprender el flujo de su programa y a identificar los errores.

☒ Utilización de un entorno de desarrollo integrado (IDE) con herramientas de depuración.

☒ **Correcto**

Correcto Los IDE suelen disponer de potentes herramientas de depuración que pueden ayudarle a recorrer el código e inspeccionar las variables.

☒ Utilizar recursos en línea como Stack Overflow para obtener ayuda.

☒ **Correcto**

Correcto Los recursos en línea pueden ofrecer soluciones a problemas comunes y ayudarte a aprender de la experiencia de otros.

4. Estás trabajando en un proyecto de software y tu programa se bloquea inesperadamente. Quieres utilizar un enfoque sistemático para resolver el problema. ¿Cuál es la acción inicial más importante de la siguiente lista? Selecciona la mejor respuesta.

- ☐ Compruebe los archivos de registro del programa en busca de cualquier actividad o entrada inusual.
- ☐ Acuda a un colega con más experiencia para que le ayude a resolver el problema.
- ☒ Revise cuidadosamente los mensajes de error generados por el programa.
- ☐ Empieza inmediatamente a modificar las secciones del código que sospechas que pueden estar causando problemas.

✔ **Correcto**

Correcto El análisis de los mensajes de error puede proporcionar pistas valiosas sobre el origen del fallo y guiar sus esfuerzos de depuración.

5. Cuando encuentras un `SyntaxError` en tu código Python, ¿cuál de las siguientes puede ser la causa? Seleccione todas las que correspondan.

- ☒ El nivel de sangría de su código.

✔ **Correcto**

Correcto Una sangría incorrecta puede causar un `SyntaxError`.

- ☒ Si algún paréntesis no está cerrado.

✔ **Correcto**

¡Correcto! Abrir paréntesis pero no cerrarlos puede causar un `SyntaxError`.

- ☐ La versión de Python que está utilizando.

- ☐ Si todas las variables se inicializan antes de su uso.

ACTIVIDAD FINAL: TRATAMIENTO DE ERRORES Y DEPURACIÓN

1. ¿Cuál de las siguientes opciones describe mejor el paso inicial en el proceso de depuración de impresión? Seleccione la mejor respuesta.

- ☒ Identifique las secciones de su código en las que sospecha que puede haber errores.
- ☐ Inserte instrucciones de impresión en todo el código.
- ☐ Ejecute su código y observe la salida.
- ☐ Analice la salida generada por las sentencias print.

☒ **Correcto**

Correcto El primer paso en la depuración de impresión consiste en revisar cuidadosamente el código y localizar las áreas que pueden estar causando problemas.

2. Estás usando un depurador interactivo para solucionar problemas de un programa Python. Has establecido un punto de interrupción y el programa se ha detenido en esa línea. ¿Cuál es el siguiente paso lógico en el proceso de depuración? Seleccione la mejor respuesta.

- ☐ Finaliza la sesión de depuración y vuelve a empezar.
- ☐ Continúe ejecutando el programa sin más inspecciones.
- ☒ Inspeccione los valores de las variables y la pila de llamadas para comprender el estado actual del programa.
- ☐ Modifique el código directamente en el depurador para solucionar el error.

☒ **Correcto**

Correcto Una vez que el programa se detiene en un punto de interrupción, el siguiente paso es examinar los valores de las variables relevantes y la pila de llamadas para obtener información sobre el estado actual del programa e identificar posibles problemas.

3. Estás trabajando en un gran proyecto Python y quieres seguir la secuencia de eventos mientras se ejecuta tu programa, incluyendo llamadas a funciones, asignaciones de variables y errores potenciales. También necesitas la capacidad de filtrar y analizar esta información más tarde, incluso después de que el programa haya terminado de ejecutarse. ¿Qué técnica de depuración se ajusta mejor a estos requisitos? Seleccione la mejor respuesta.

- ☐ Afirmaciones
- ☐ Depurador interactivo
- ☒ Registro
- ☐ Imprimir declaraciones

logging

✓ **Correcto**

Correcto El registro proporciona una forma estructurada de registrar eventos, errores y otra información durante la ejecución del programa, permitiendo su posterior análisis y filtrado.

4. ¿Qué técnica de depuración consiste en explicar tu código línea por línea a un objeto inanimado para ayudarte a identificar errores? Seleccione la mejor respuesta.

- ☐ Depuración de impresión
- ☐ Pruebas unitarias
- ☒ Depuración del patito de goma
- ☐ Comentarios de código

✓ **Correcto**

Correcto La depuración con patito de goma es una técnica que consiste en explicar el código en voz alta, lo que a menudo ayuda a detectar errores que podrían haberse pasado por alto.

5. ¿Cuál de los siguientes es un ejemplo de una excepción común de Python que indica un intento de dividir un número por cero? Seleccione la mejor respuesta.

- ☐ FileNotFoundError
- ☐ ValueError
- ☐ TypeError
- ☒ ZeroDivisionError

✓ **Correcto**

Correcto Esta excepción se produce específicamente cuando se intenta dividir un número por cero.

6. Estás trabajando en un proyecto de Python que implica la lectura de datos de archivos y la realización de cálculos. ¿Cuál de las siguientes excepciones es más probable que encuentres en un proyecto de este tipo? Selecciona todas las que correspondan.

☒ FileNotFoundError

☒ **Correcto**

Correcto Esta excepción se produce cuando se intenta acceder a un archivo que no existe o no se puede encontrar en la ubicación especificada.

☒ ValueError

☒ **Correcto**

Correcto Esta excepción puede surgir si se intenta realizar una operación con un valor que es del tipo correcto pero tiene un valor inapropiado, como por ejemplo intentar convertir una cadena no numérica a un entero.

☒ ZeroDivisionError

☒ **Correcto**

Correcto Si tus cálculos implican división, existe la posibilidad de que accidentalmente intentes dividir por cero, provocando esta excepción.

☐ `NombreError

7. Estás escribiendo un programa Python que lee datos de un fichero llamado input.txt. ¿Cómo usarías un bloque try-except para manejar el escenario en el que el archivo podría no existir? Selecciona la mejor respuesta.



```
1 with open("input.txt", "r") as file:
2     try:
3         data = file.read()
4     except FileNotFoundError:
5         print("The file 'input.txt' was not found.")
```



```
1 try:
2     data = open("input.txt", "r").read()
3 except FileNotFoundError:
4     print("The file 'input.txt' was not found.")
```



```
1 try:
2     with open("input.txt", "r") as file:
3         data = file.read()
4 except:
5     print("An error occurred.")
```



```
1 try:
2     with open("input.txt", "r") as file:
3         data = file.read()
4 except FileNotFoundError:
5     print("The file 'input.txt' was not found.")
```

✔ **Correcto**

Correcto Este código utiliza correctamente un bloque `try-except` para manejar el `FileNotFoundError` que puede ocurrir si el fichero no existe. La sentencia `with` asegura que el fichero se cierra correctamente, incluso si se produce un error.

8. ¿El Manejo de excepciones de Python puede incluir cuál de las siguientes palabras clave? Seleccione todas las que correspondan.

☐ capture

☒ finalmente

✔ **Correcto**

Correcto La palabra clave `finally`, aunque opcional, se utiliza para denotar código que se ejecutará, independientemente de si se ha lanzado una excepción o no.

☒ pruebe

✔ **Correcto**

Correcto La palabra clave `try` es el campo de pruebas para su código.

☒ excepto

✔ **Correcto**

Correcto La palabra clave `except` permite especificar la excepción a atrapar y las acciones a realizar.

9. Estás desarrollando un módulo de Python que proporciona varias funciones matemáticas. Quieres asegurarte de que los usuarios de tu módulo reciben mensajes de error claros e informativos cuando introducen datos no válidos en tus funciones. ¿Cuál es el método recomendado para conseguirlo? Seleccione la mejor respuesta.

- ☐ Captura excepciones utilizando una cláusula except: y proporciona un mensaje de error genérico.
- ☐ Ignora silenciosamente las entradas no válidas y devuelve un valor por defecto.
- ☒ Defina excepciones personalizadas para condiciones de error específicas en su módulo y levántelas con mensajes descriptivos.
- ☐ Confía en las excepciones incorporadas en Python y sus mensajes de error por defecto.

✓ **Correcto**

Correcto Este enfoque le permite adaptar los mensajes de error al contexto específico de sus funciones matemáticas, proporcionando a los usuarios información más útil.

10. Estás trabajando en un programa Python, e inesperadamente se bloquea con un mensaje TypeError que indica que "el objeto int no es iterable". ¿Qué indica este mensaje de error? Selecciona la mejor respuesta.

- ☐ Está intentando realizar una operación con tipos de datos incompatibles
- ☐ Está intentando utilizar una variable que no ha sido definida
- ☒ Estás intentando iterar sobre un entero como si fuera una lista u otro tipo iterable.
- ☐ Estás intentando dividir un entero por cero

✓ **Correcto**

Correcto Este error significa que está intentando utilizar un número entero en un contexto que espera algo que se pueda recorrer en bucle, como una lista o una cadena.